

Apollo247 Selenium BDD Automation Framework Documentation

1. Project Title

Apollo247 Healthcare Web Application Automation Testing using Selenium with Behave BDD Framework

2. Project Description

This project automates the Apollo247 healthcare web application using Selenium WebDriver integrated with the Behave BDD framework. The framework follows the Page Object Model (POM) architecture to improve maintainability, scalability, and reusable test design.

- BDD implementation using Behave and Gherkin syntax.
- Reusable Page Object Model structure.
- Automated doctor search and appointment booking workflows.
- CSV and Excel-based external test data handling.
- Logging and reporting integration using Allure.
- Cross-browser capable Selenium automation structure.

3. Project Architecture

- features/: Contains Gherkin feature files.
- features/steps/: Step definitions implementation.
- pages/: Page Object Model classes.
- utils/: Utility modules for configuration, drivers, retry, CSV, and Excel readers.
- config/: Environment and configuration settings.
- reports/: Stores generated reports and logs.

4. Test Strategy

The automation framework applies a Behavior Driven Development strategy where business scenarios are written in Gherkin language and implemented through Selenium automation.

Strategy Area	Implementation
Framework Type	BDD with Behave
Design Pattern	Page Object Model (POM)
Automation Scope	UI Functional Testing
Validation	Assertions and element verification
Data Handling	CSV and Excel data-driven testing
Execution	Command-line and batch execution
Reporting	Allure Reports

5. Test Environment

Component	Details
Programming Language	Python
Automation Tool	Selenium WebDriver
BDD Framework	Behave
IDE	PyCharm
Browser	Google Chrome
Reporting Tool	Allure Reports
Operating System	Windows
Dependency Management	requirements.txt

6. Test Execution

The test suite is executed using Behave commands integrated with Allure reporting.

- behave
- behave -f allure_behave.formatter:AllureFormatter -o reports/allure-results --no-capture
- python runtests.py

7. Test Scenarios Covered

- User launches Apollo247 application.
- User navigates to Find Doctors module.
- Doctor search functionality validation.
- Doctor availability validation.
- Appointment booking workflow.
- Positive and workflow-based functional scenarios.

8. Test Output

The framework generates execution logs, screenshots, and Allure-compatible result files after execution.

- Console execution logs.
- Step-level execution details.
- Screenshot capture during failures.
- Allure result JSON files.
- Execution status for passed and failed scenarios.

9. Test Report Analysis

Allure reports provide detailed execution analytics for the BDD automation suite.

- Scenario-wise pass/fail status.
- Execution timeline and duration.
- Detailed step execution tracking.
- Failure screenshots and logs.
- Traceability between feature files and step definitions.
- Improved debugging and root-cause analysis.

10. Project Strengths

- Clean modular architecture.
- High reusability using POM.
- Readable BDD feature scenarios.
- Scalable framework structure.
- Integrated logging and reporting.
- Externalized test data support.

11. Conclusion

The Selenium Behave BDD framework successfully automates critical workflows of the Apollo247 application. The framework is scalable, maintainable, and suitable for enterprise-level automation execution with detailed reporting and reusable components.